ASSESSMENT APPROACH

BookLeaf Author Support

Ayush Srivastava | Full Stack Developer Assessment

What I prioritized

I prioritized the complete support loop: an author understands their book and royalties, raises a contextual query, and receives a reviewed staff response without refreshing. The author dashboard emphasizes publishing facts; the staff inbox places the conversation, reply editor, AI suggestions, and evidence together. The hosted demo uses all 10 authors and 18 books from the supplied dataset.

Architecture and trade-offs

I chose Next.js and TypeScript for a single deployable frontend and REST API, with PostgreSQL and Drizzle for explicit relationships, transactions, and migrations. Better Auth handles sessions; server-side profiles determine roles and every author query is scoped to ownership. Private notes and activity history are staff-only. This keeps the assessment straightforward to run while separating interface components, API handlers, persistence, and AI logic.

Five-second polling meets the near-real-time requirement without a separate connection service. It trades extra requests and a short update delay for simpler deployment. Version checks protect concurrent edits and stale AI drafts; operation IDs make repeated submissions safe. The attachment selector remains UI-only, as permitted by the brief.

AI as a reviewed support tool

OpenRouter routes requests to GPT-4.1 mini. I chose this model for structured output support and the cost of a bounded classification-and-drafting task. Tickets are saved before classification so AI failure does not lose a query. Staff can override category and priority, inspect policy sources, edit the draft, and send manually when AI is unavailable.

Draft context includes selected policies, relevant book facts, and at most three recent public replies. Contact fields and internal notes are excluded; recognizable personal identifiers in free text are redacted. Structured validation, timeouts, caching, and daily usage limits control failures and cost. Cumulative royalties cannot establish an overdue payment or bank verification; the assistant must identify missing evidence rather than invent it.

Verification and production evolution

Checks cover ownership boundaries, private data, duplicate requests, concurrent updates, stale drafts, invalid model output, and manual fallback. Hosted browser checks cover both roles, four viewport sizes, reply polling, session recovery, and a synthetic AI-assisted ticket flow.

For production, I would add durable AI jobs with idempotent retries, private account provisioning, monitoring and retention controls, and verified publishing/payment integrations. I would evaluate policy adherence, escalation accuracy, latency, cost, and staff edits before expanding automation.